Treap树

一、引言

1.1 什么是Treap?

Treap（Tree+Heap）是一种随机化的二叉搜索树，结合了**二叉搜索树（BST）** 和**堆（Heap）**的特性：

- **BST性质**：每个节点的左子树所有节点值（也被称为“键值”）小于该节点，右子树所有节点值大于该节点
- **堆性质**：每个节点有一个随机分配的优先级，父节点的优先级小于（或大于）子节点的优先级

1.2 为什么需要Treap?

- 普通BST在最坏情况下可能退化成链表（O(n)操作复杂度）
- Treap通过随机优先级维持树的平衡性，**期望高度为O(log n)**
- 实现相对简单，无需复杂的旋转操作（如AVL树、红黑树）

二、Treap的核心思想

2.1 两种维护方式

1. **旋转Treap**：使用旋转操作维护堆性质
2. **无旋转Treap（FHQ Treap）**：使用分裂合并操作，代码更简洁

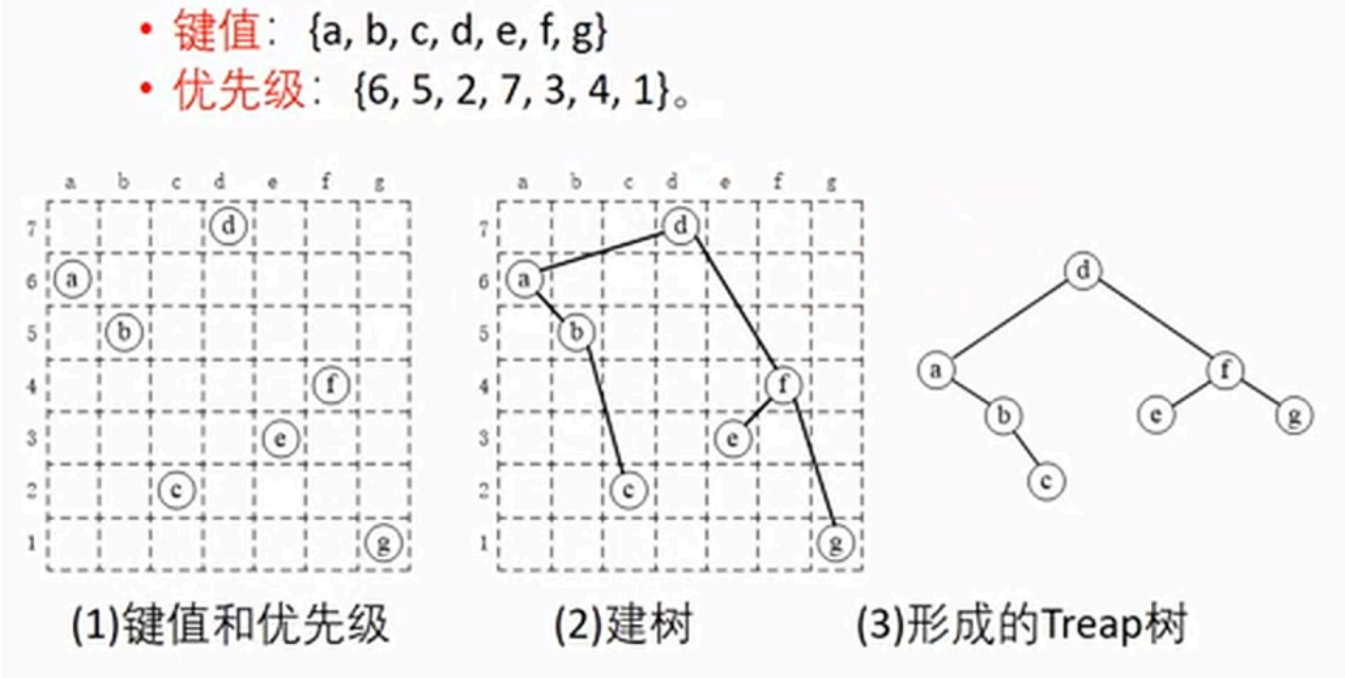
2.2 关键特性

- **随机化**：每个节点随机分配优先级，使树保持平衡的概率很高
- **期望复杂度**：所有基本操作（插入、删除、查找）的期望时间复杂度为**O(log n)**

三、Treap树的性质与随机化平衡原理

3.1 形态唯一性

若每个节点的键值、优先级已经事前确定而且不同，那么建立的BST的形态是唯一的，与节点的插入顺序没有关系。可以把每个点的（键值，优先级）看作它在平面上的坐标(x,y)，坐标确定了它的位置。若优先级是随机产生的，那么在概率上就实现了二叉树的平衡。



3.2 随机化平衡

Treap树如何解决平衡问题?
合理分配结点的优先级，可以得到一个比较平衡的BST。
一个简单的分配方法:随机。对每个结点的优先级进行随机赋值，生成的Treap树的形态也是随机的。虽然不能保证每次生成的Treap树是平衡的，但是期望的插入、删除、查找的时间复杂度都是0(logn)的。

如果预先知道所有节点的键值，那么建树很简单：先按键值排序，然后从键值最小的开始，从左到右逐个向树上加入节点，加入时按优先级（或者已知，或者随机生成）在纵向上调整形态，这就是笛卡尔树，它的建树复杂度为O(n)。

更常见的情况是需要动态加入新的节点，并不能预先知道键值和优先级。做法是每读入一个新键值，为它分配一个随机的优先级，插入树中，插入时动态调整树的结构，使它仍是一颗Treap

树。此时建一棵n个节点的树，复杂度为O(nlogn)。

如何调整和维护Treap树？有两种方法：①旋转法；②FHQ。两种方法的计算复杂度都为O(nlogn)。旋转法是经典方法，FHQ比旋转法编码更简单，而且能用于区间翻转、移动、持久化等场合。

四、数据结构

4.1 节点定义

```
// 节点结构定义
struct Node{
    int ls, rs;   // 左右儿子
    int key, pri; // key：键值；pri：随机的优先级
    int size;     // 当前结点为根的子树的结点数量，用于求第k大和rank
} t[M];           // tree[]，存树
```

4.2 初始化与更新

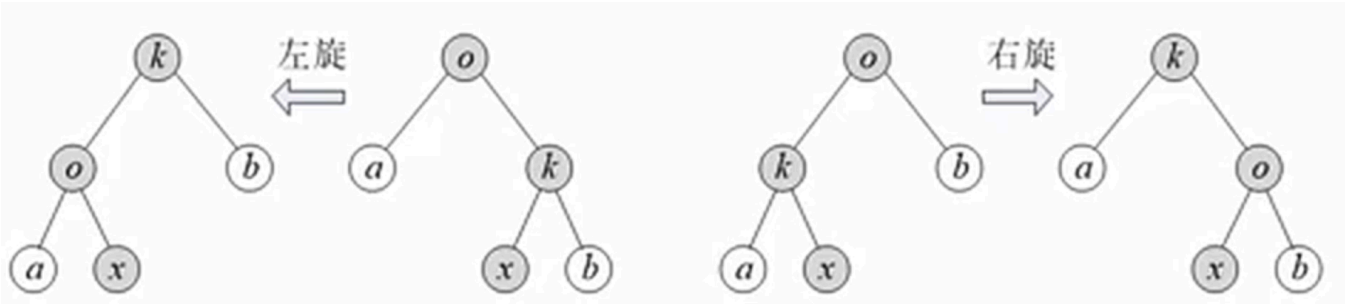
```
void newNode(int x){          // 初始化一个新结点
    cnt++; // 从t[1]开始存储结点，t[0]被放弃。若子结点是0，表示没有子结点
    t[cnt].size = 1;
    t[cnt].ls = t[cnt].rs = 0; // 0表示没有子结点
    t[cnt].key = x;            // key: 键值
    t[cnt].pri = rand();       // pri: 随机产生，优先级
}
void Update(int u){ // 更新以u为根的子树的size
    t[u].size = t[t[u].ls].size + t[t[u].rs].size + 1;
}
```

五、基于旋转法的Treap树操作

5.1 插入节点

把新结点x插入到Treap树，分两步:

(1)先把x按键值大小插入到合适的子树上。
(2)给x随机分配一个优先级，如果x的优先级违反了堆的性质，即它的优先级比父结点高，那么进行调整，让x往上走，替代父结点，最后得到一个新的Treap树。

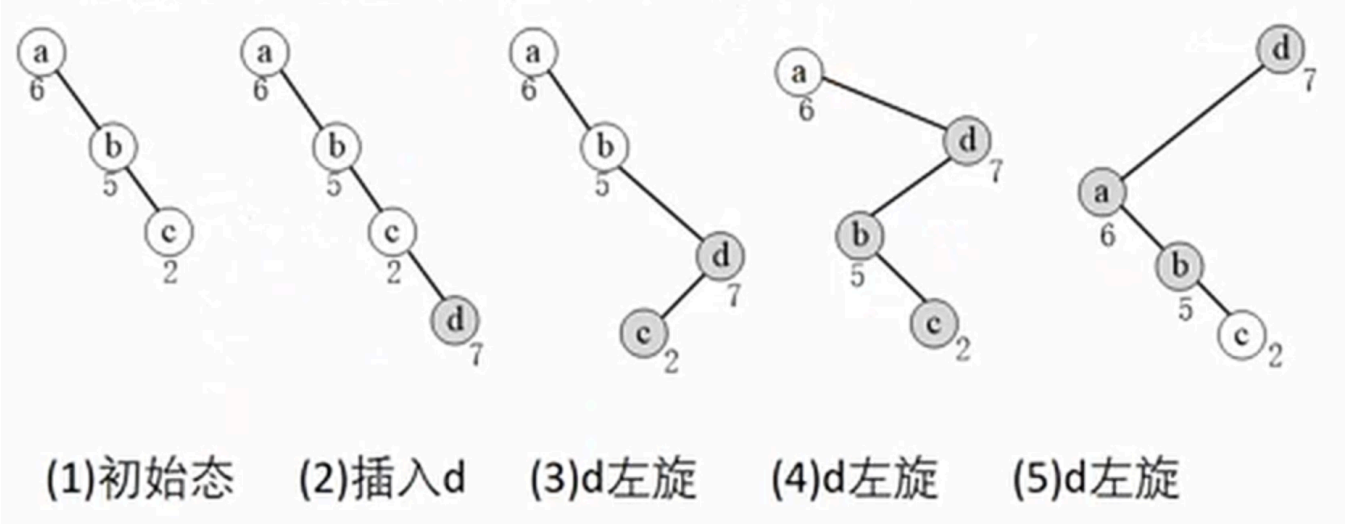


旋转法插入节点的代码如下，代码中定义的节点名称o、k与上图的节点名称对应。

```
void rotate(int &o, int d){ // 旋转， d=0右旋，d=1左旋
    int k;
    if (d == 1){ // 左旋，把o的右儿子k旋到根部
        k = t[o].rs;
        t[o].rs = t[k].ls; // 图中的x
        t[k].ls = o;
    }else{ // 右旋，把o的左儿子k旋到根部
        k = t[o].ls;
        t[o].ls = t[k].rs; // 图中的x
        t[k].rs = o;
    }
    t[k].size = t[o].size;
    Update(o);
    o = k; // 新的根是k
}

```

用一个例子说明新结点的插入和调整的过程。



图(1)是初始状态的BST{a,b,c}
图(2)插入d点，按朴素的插入方法插入到底部;
图(3)d的优先级比父结点c高，左旋，上升;
图(4)d的优先级比新的父结点b高，继续左旋上升;
图(5)再次左旋上升，完成了新的Treap树。

5.2 删除节点

待删除的结点x是叶子结点:直接删除。
待删除的结点x有子结点:找到优先级最大的子结点，把x向相反的方向旋转，也就是把x向树的下层调整，直到x被旋转到叶子结点，然后直接删除。

5.3 排名

“排名”和“求第k大数”一般被称为“名次树问题“，是BST的典型应用，在任何一种BST上都容易用递归遍历得到，Treap树也常用了求名次。它们都用到了节点的size值，即以这个节点为根的子树上的节点总数。

求数字x的排名。从根节点开始递归查找，递归到节点u时：若u的键值大于或等于x，x在u的左子树上，继续递归u的左子树；若u的键值小于x，x在u的右子树上，递归u的右子树，并加上u的左子树的size值。

5.4 求第k大数

根据节点的size值不断递归整颗树，求得第k大数。

5.5 前驱和后继

前驱是求比x小的数，后继是求比x大的数，计算过程与排名的过程类似。

5.6 分裂与合并

把一棵树分裂成两棵树，或者把两棵树合并成一颗。
Treap树做这样的操作，比较繁琐。

一般用Splay树做分裂与合并。

六、例题 P3369 【模板】普通平衡树

题目描述

您需要动态地维护一个可重集合 M，并且提供以下操作：

1. 向 M 中插入一个数 x。
2. 从 M 中删除一个数 x（若有多个相同的数，应只删除一个）。
3. 查询 M 中有多少个数比 x 小，并且将得到的答案加一。
4. 查询如果将 M 从小到大排列后，排名位于第 x 位的数。
5. 查询 M 中 x 的前驱（前驱定义为小于 x，且最大的数）。
6. 查询 M 中 x 的后继（后继定义为大于 x，且最小的数）。

对于操作 $3, 5, 6$，**不保证**当前可重集中存在数 x。

对于操作 $5, 6$，保证答案一定存在。

输入格式

第一行为 n，表示操作的个数，下面 n 行每行有两个数 opt 和 x，opt 表示操作的序号（$1 \leq \text{opt} \leq 6$）。

输出格式

对于操作 $3, 4, 5, 6$ 每行输出一个数，表示对应答案。

输入输出样例 #1

输入 #1

```
10
1 106465
4 1
1 317721
1 460929
1 644985
1 84185
1 89851
6 81968
1 492737
5 493598
```

输出 #1

```
106465
84185
492737
```

说明/提示

【数据范围】

对于 100% 的数据，$1 \le n \le 10^5$，$|x| \le 10^7$。